BugBoard26

Documentazione finale

Requisiti, design e testing

Progetto di Ingegneria del Software
A.A. 2025/2026 – Canale 2

Componenti	Valerio Porcile – N86004517
	Emanuel Giuseppe Lupoli – N86004280
	Emanuele Martucci – N86004084

Modalita'	Tradizionale
Versione	1.0 – consolidamento per la consegna
Data di riferimento	16 ottobre 2026
Codice esaminato	`e139951`

Stato del documento

Questo documento riunisce la specifica dei requisiti, il design del sistema e le attivita' di testing in capitoli distinti, secondo la sezione 8 della traccia ufficiale v1.1. Riprende il materiale dei due homework e lo confronta con il codice disponibile nel repository.

Le funzionalita' descritte sono presenti nel codice. Il collaudo frontend con API e PostgreSQL reali del 16 ottobre 2026 e' riportato nel capitolo 3. Questo non equivale alla conclusione di tutte le attivita' di consegna: restano da consolidare la suite backend completa, il report SonarQube, le prove di persistenza dopo ricreazione dei container, la valutazione di usabilita' e i chiarimenti indicati nel capitolo 4. Non vengono attribuiti esiti positivi alle attivita' non eseguite.

Repository: `https://github.com/Valeriop03/BugBoard26`. La traccia di riferimento e' il file `Progetto-SWE-2025-2026-canale2-v1.1.pdf` nella radice. I sorgenti dei homework sono conservati come materiale precedente; questo documento costituisce la versione unificata da completare con gli ultimi risultati prima della consegna.

Indice

Stato del documento **i**

1 Specifica dei requisiti **1**
1.1 Scopo e perimetro . . . 1
1.2 Glossario e utenti target . . . 2
1.3 Requisiti funzionali . . . 2
1.4 Requisiti non funzionali e vincoli . . . 3
1.5 Modello dei casi d'uso . . . 3
1.6 UC-01 – Segnalare una issue . . . 5
1.7 UC-02 – Risolvere una issue assegnata . . . 6
1.8 UC-03 – Contrassegnare un duplicato . . . 7

2 Design del sistema e del software **8**
2.1 Architettura e scelte tecnologiche . . . 8
2.2 Organizzazione Object-Oriented . . . 11
2.3 Persistenza . . . 14
2.4 Autenticazione e autorizzazione . . . 16
2.5 Ricerca, suggerimento ed export . . . 16
2.6 Immagini, archivio e duplicati . . . 16
2.7 Diagrammi comportamentali . . . 16
2.8 Interfacce e avvio . . . 20
2.9 Versionamento e artefatti . . . 20

3 Testing e valutazione dell'usabilita' **22**
3.1 Strategia e livelli di verifica . . . 22
3.2 Tre metodi di dominio e classi di equivalenza . . . 22
3.3 Test plan – Risoluzione di una issue . . . 23
3.4 Collaudo frontend reale – 16 ottobre 2026 . . . 23
3.5 Esecuzione ripetibile e qualita' backend . . . 24
3.6 Valutazione dell'usabilita' – piano da eseguire . . . 25
3.6.1 Review tramite checklist . . . 25
3.6.2 Sessioni con utenti e questionario . . . 25

4 Limiti e completamento della consegna **26**
4.1 Punti ancora aperti . . . 26
4.2 Pacchetto e discussione . . . 26
4.3 Manutenzione del documento . . . 27

Capitolo 1

Specifica dei requisiti

1.1 Scopo e perimetro

BugBoard26 permette a un gruppo di lavoro di segnalare e consultare issue, seguirne lo stato e comunicare la risoluzione al segnalatore. Il progetto e' una web app distribuita con frontend e backend indipendenti. Le funzionalita' assegnate sono esclusivamente quelle riportate sotto.

N. traccia	Funzionalita' assegnata
1	Login email/password, amministratore iniziale e creazione delle utenze da parte di ADMIN.
2	Creazione issue con titolo, descrizione, tipo, priorita' e immagine opzionali; stato iniziale TODO.
3	Lista riepilogativa, filtri e ordinamento.
6	Cambio stato da parte dell'assegnatario; notifica al segnalatore alla risoluzione.
8	Esportazione dell'elenco in formato condivisibile, realizzata con CSV.
11	Ricerca per parole chiave.
13	Archiviazione riservata ad ADMIN e consultazione in una pagina dedicata.
14	Suggerimento dell'assegnatario in base al carico di lavoro corrente.
15	Utenze READONLY create dagli amministratori per la consultazione.
16	Contrassegno duplicato riservato ad ADMIN e chiusura logica immediata.

Non sono incluse gestione dei commenti, dashboard, tag, cronologia, report mensili o scadenze. Il campo assegnatario e' necessario per il cambio stato e il suggerimento, ma non introduce il flusso separato di assegnazione e relativa notifica della funzionalita' 4, non assegnata. La notifica di risoluzione e' interna all'applicazione, non un'email.

1.2 Glossario e utenti target

Termine	Significato
Issue	Segnalazione di tipo QUESTION, BUG, DOCUMENTATION o FEATURE.
Segnalatore	Utente che ha creato la issue.
Assegnatario	Utente indicato nel campo `AssignedToId`.
Archiviata	Issue esclusa dalla lista principale, ma conservata e consultabile.
Duplicata	Issue chiusa logicamente come DUPLICATE e collegata a un originale.
Notifica	Messaggio persistente per un singolo destinatario, con stato di lettura.

Gli utenti target sono membri di un team di sviluppo e referenti esterni. I primi devono registrare e consultare segnalazioni durante il lavoro; i secondi devono seguirne l'avanzamento senza modificare le issue. Non e' previsto l'accesso anonimo ai dati del progetto.

Attore	Responsabilita'
ADMIN	Crea utenti e issue, consulta i dati, cambia stato anche senza assegnazione, archivia e marca duplicati.
USER	Crea e consulta issue; cambia stato soltanto se assegnatario.
READONLY	Consulta issue, dettagli, immagini e archivio; usa ricerca, filtri ed export senza modificare issue.

Tutti i ruoli autenticati possono consultare il suggerimento assegnatario e le proprie notifiche. Il codice permette anche a READONLY di segnare le proprie notifiche come lette: la compatibilita' con RF-17 resta da chiarire, come indicato nel capitolo 4.

1.3 Requisiti funzionali

Gli identificativi seguenti sono mantenuti dalla specifica dei requisiti; non coincidono con i numeri delle funzionalita' nella traccia.

ID	Requisito
RF-01	Login tramite email e password.
RF-02	Account amministratore iniziale gia' attivo.
RF-03	Creazione utenti da ADMIN con email, password e ruolo.
RF-04	Creazione issue da utente autenticato non READONLY con titolo e descrizione.
RF-05	Tipo della issue, priorita' opzionale e immagine opzionale.
RF-06	Stato iniziale TODO.
RF-07	Lista delle issue non archiviate.
RF-08	Filtri e ordinamento per criteri rilevanti.
RF-09	Ricerca per parole chiave su titolo e descrizione.
RF-10	Cambio stato da parte dell'assegnatario; ADMIN ha accesso completo.

ID	Requisito
RF-11	Notifica al segnalatore quando la issue passa a RESOLVED.
RF-12	Esportazione dell'elenco in CSV.
RF-13	Archiviazione riservata ad ADMIN.
RF-14	Esclusione degli archiviati dalla lista principale.
RF-15	Consultazione delle issue archiviate in una pagina dedicata.
RF-16	Suggerimento assegnatario basato sul carico corrente.
RF-17	READONLY visualizza issue e dettagli senza modificare dati.
RF-18	ADMIN puo' contrassegnare una issue come duplicata di un'altra.
RF-19	Chiusura immediata della issue contrassegnata come duplicata.

1.4 Requisiti non funzionali e vincoli

ID	Requisito
RNF-01	Password memorizzate tramite hashing, non in chiaro.
RNF-02	Verifica dei permessi nelle API, non soltanto nell'interfaccia.
RNF-03	Frontend e backend separati; comunicazione esclusivamente via API REST.
RNF-04	Stato persistente gestito centralmente dal backend e dal database.
RNF-05	Interfaccia leggibile, con operazioni disponibili coerenti con il ruolo.
RNF-06	Lista consultabile fluidamente con i dati dimostrativi; nessuna prestazione su larga scala dichiarata senza misure.
RNF-07	Separazione tra controller, servizi, modelli e accesso ai dati.
RNF-08	Avvio locale ripetibile seguendo il README.

Il codice usa linguaggi Object-Oriented e non delega logica e persistenza a servizi MBaaS. La traccia richiede strumenti CASE per il design. Il deployment su cloud e' auspicato, ma non obbligatorio: non costituisce una funzionalita' mancante del progetto.

1.5 Modello dei casi d'uso

Il diagramma rappresenta i ruoli e le funzionalita' assegnate. Il login precede le operazioni protette. La notifica e' legata alla risoluzione; la chiusura del duplicato non richiede un comando separato.

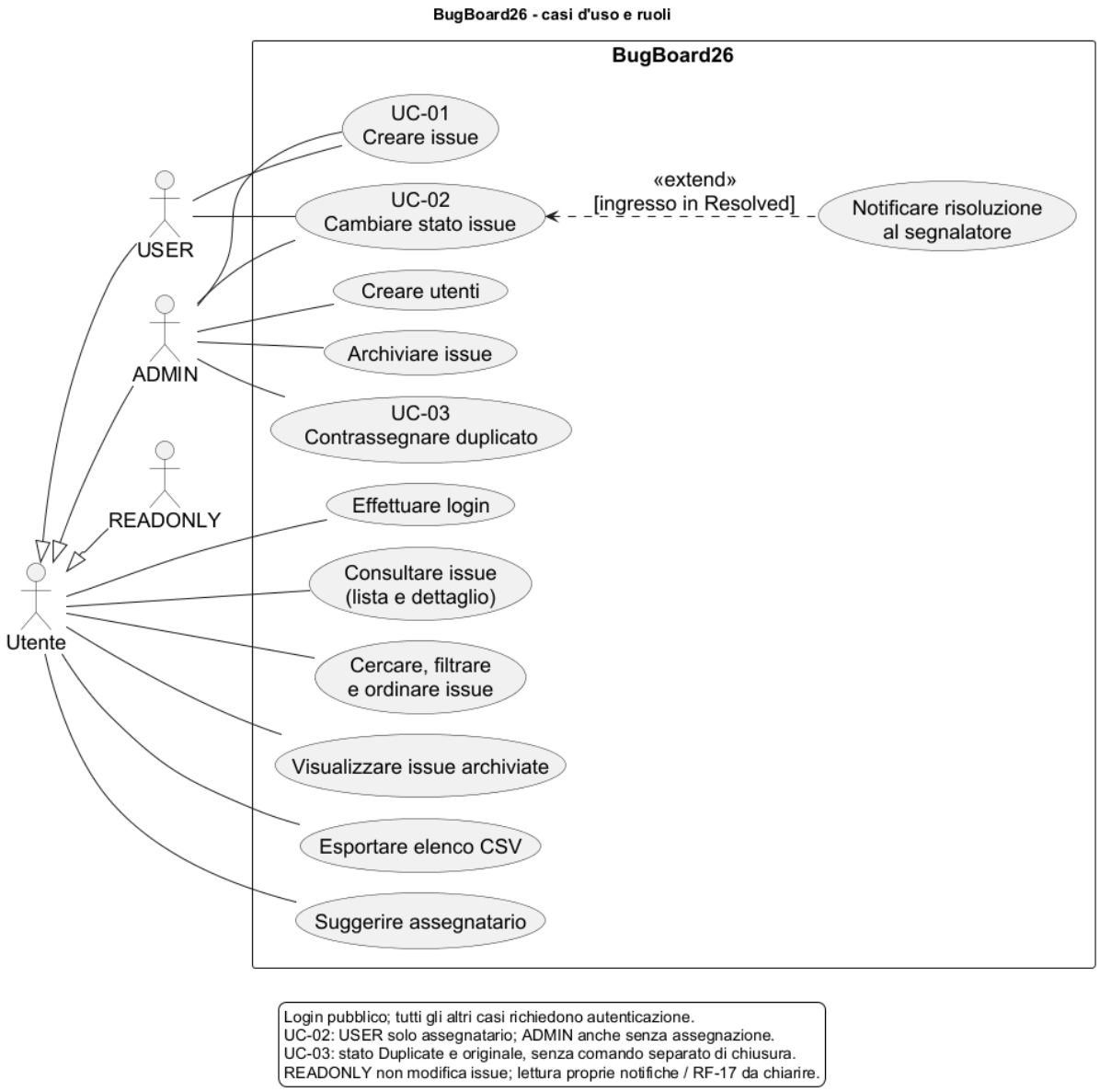


Figura 1.1: Diagramma dei casi d'uso.

1.6 UC-01 – Segnalare una issue

Campo	Descrizione
Attore e scopo	USER oppure ADMIN registra una nuova segnalazione.
Precondizioni	Account attivo e autenticato, non READONLY.
Postcondizione	Issue persistita in TODO e consultabile nella lista e nel dettaglio.
Scenario principale	1. L'utente apre il form. 2. Inserisce titolo, descrizione e tipo; puo' scegliere priorita' e immagine. 3. Il sistema mostra il suggerimento assegnatario. 4. Valida i dati e crea la issue. 5. Se selezionata, carica l'immagine. 6. Apre il dettaglio della issue creata.
Alternative	Dati mancanti/non validi: nessuna creazione. Suggerimento indisponibile: creazione senza assegnatario. Upload fallito dopo creazione: issue conservata, possibilita' di riprovare solo l'upload o aprire il dettaglio senza immagine.
Requisiti	RF-04, RF-05, RF-06, RF-07, RF-16.

Il form richiede almeno tre caratteri dopo la rimozione degli spazi esterni per titolo e descrizione; i limiti massimi sono rispettivamente 200 e 4000 caratteri. Il suggerimento viene usato come assegnatario dal form; non e' una nuova API di assegnazione. Se indicato, l'assegnatario deve essere attivo e non READONLY. La creazione JSON e l'upload multipart sono due richieste distinte.

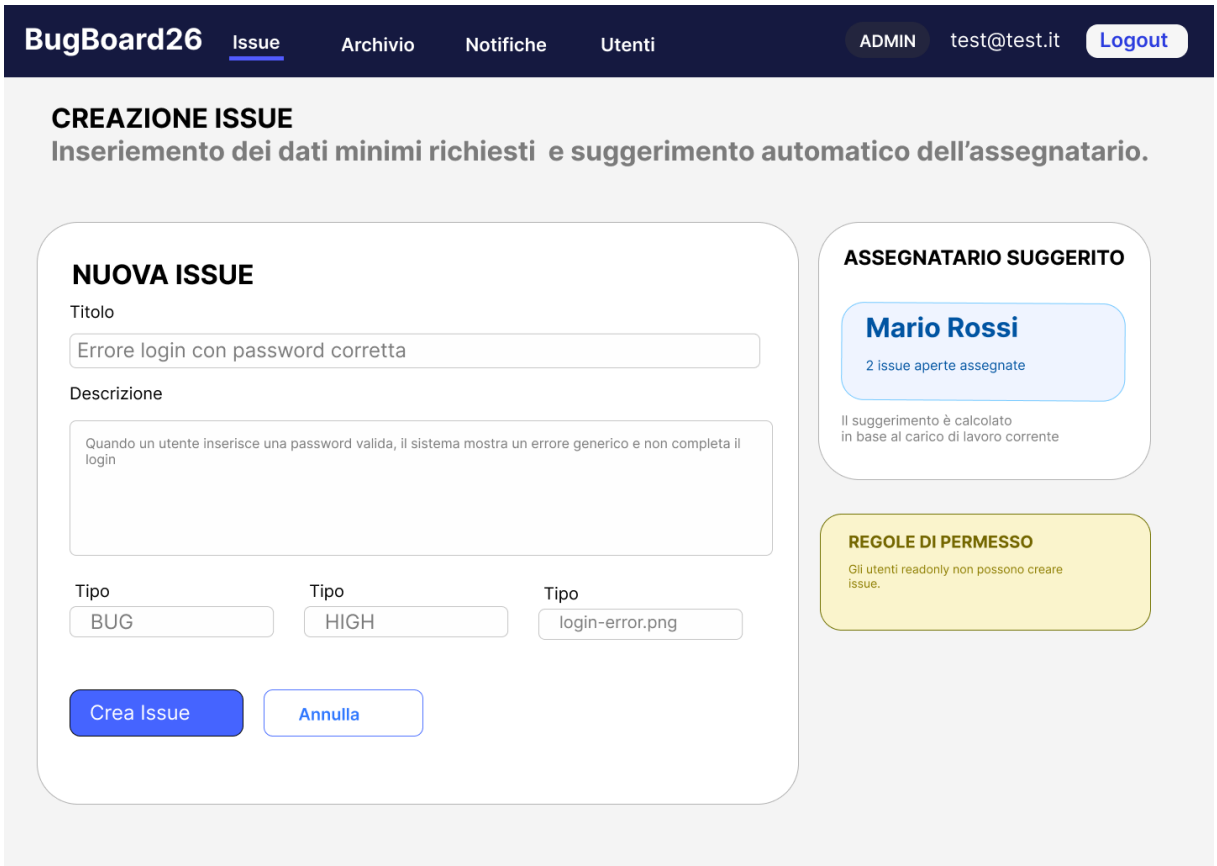


Figura 1.2: Mock-up di creazione: riferimento progettuale, non screenshot del collaudo.

1.7 UC-02 – Risolvere una issue assegnata

Campo	Descrizione
Attore e scopo	USER assegnatario oppure ADMIN aggiorna l'avanzamento della issue.
Precondizioni	Account attivo e autenticato; issue esistente e non contrassegnata come duplicata.
Postcondizioni	Nuovo stato salvato; alla risoluzione sono valorizzati data di risoluzione e notifica non letta per il segnalatore.
Scenario principale	1. L'utente apre il dettaglio. 2. Seleziona RESOLVED. 3. Il backend verifica ruolo e assegnazione. 4. Aggiorna stato e date, crea la notifica e salva. 5. Il frontend mostra il risultato aggiornato.
Alternative	USER non assegnatario o READONLY: 403. Issue inesistente: 404. Stato gia' corrente: risposta senza nuova notifica o nuove date. Issue duplicata: 409. Token non valido: 401.
Requisiti	RF-10, RF-11, RF-17, RNF-02.

Non e' imposta una matrice aggiuntiva di transizioni. Il passaggio da RESOLVED a un altro stato azzera la data di risoluzione; una risoluzione successiva genera una nuova notifica. Le notifiche precedenti restano.

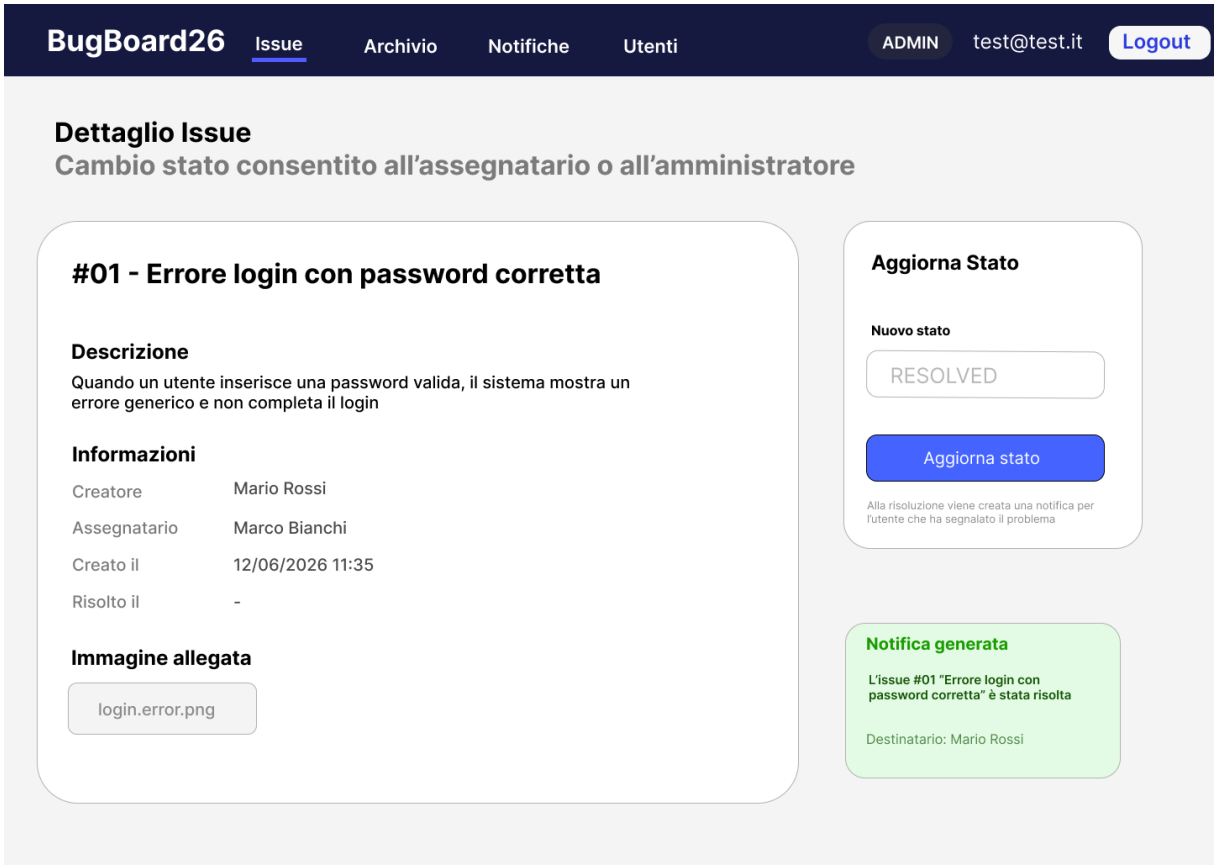


Figura 1.3: Mock-up del dettaglio e del cambio stato.

1.8 UC-03 – Contrassegnare un duplicato

Campo	Descrizione
Attore e scopo	ADMIN collega una segnalazione duplicata all'originale.
Precondizioni	Autenticazione ADMIN; entrambe le issue esistono e sono distinte.
Postcondizioni	Stato DUPLICATE, riferimento all'originale salvato, `ResolvedAt` nullo.
Scenario principale	1. Apre il dettaglio del duplicato. 2. Inserisce l'ID dell'originale. 3. Il backend verifica i riferimenti. 4. Salva stato e collegamento. 5. Il frontend aggiorna il dettaglio.
Alternative	Originale assente o autoreferenza: 400. Issue corrente assente: 404. Ruolo non ADMIN: 403.
Requisiti	RF-18, RF-19, RNF-02.

DUPLICATE rappresenta la chiusura logica, non l'eliminazione. Il percorso di cambio stato non permette di impostarlo direttamente: occorre l'API dedicata con il riferimento all'originale.

Capitolo 2

Design del sistema e del software

2.1 Architettura e scelte tecnologiche

Il frontend Angular presenta i dati e invia richieste HTTP. ASP.NET Core Web API espone le operazioni, controlla i permessi e gestisce la persistenza tramite Entity Framework Core e PostgreSQL. Il backend funziona anche senza frontend; nessuna pagina Angular accede direttamente al database o alla directory delle immagini.

Angular e' usato per routing, form e servizi HTTP. ASP.NET Core permette di organizzare endpoint, dependency injection e autorizzazione nello stesso progetto. PostgreSQL viene avviato in Docker per ridurre le differenze di configurazione fra i tre PC. Il progetto backend usa .NET 10, il frontend Angular 19; le versioni effettive dei pacchetti sono nei rispettivi file di progetto e lockfile.

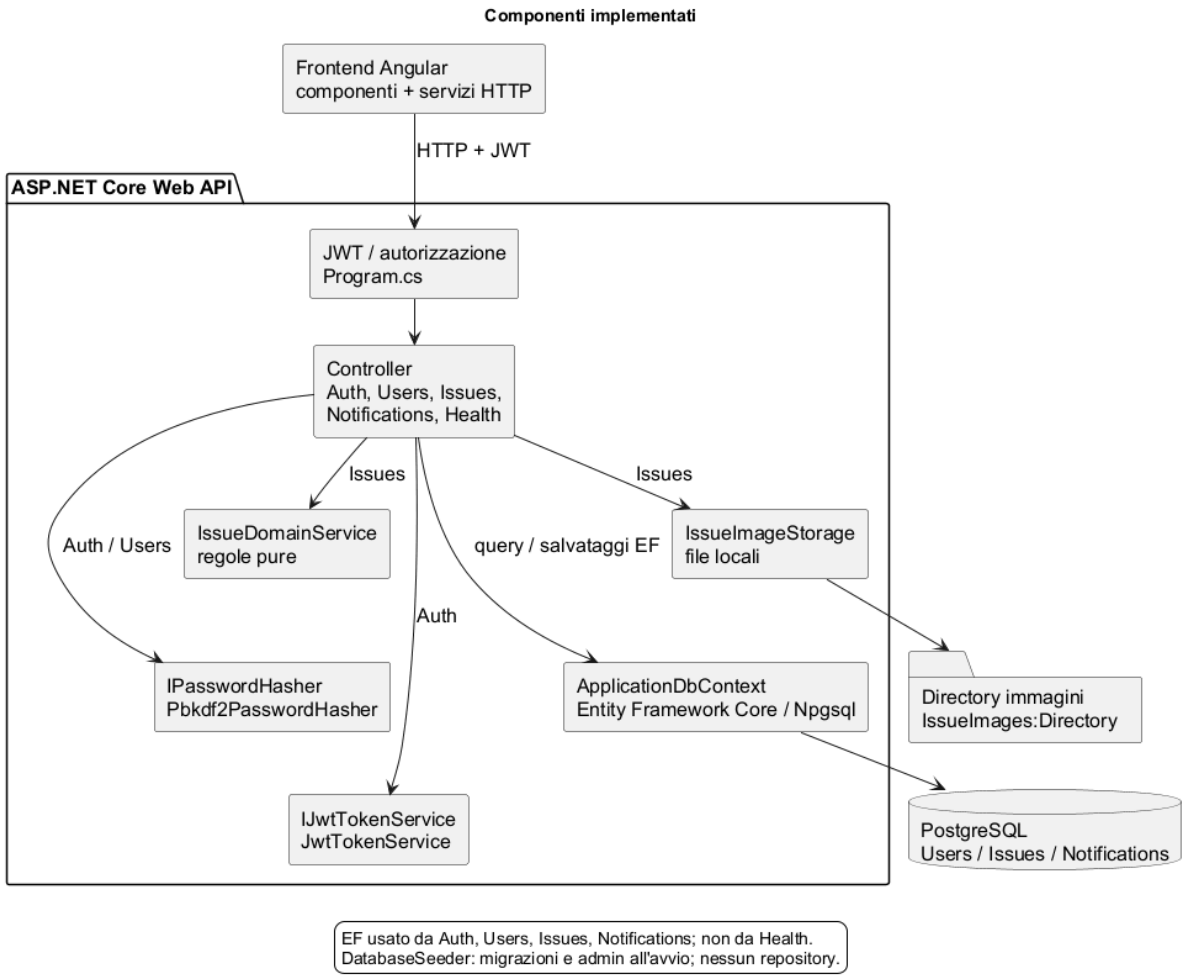


Figura 2.1: Componenti e comunicazione del sistema.

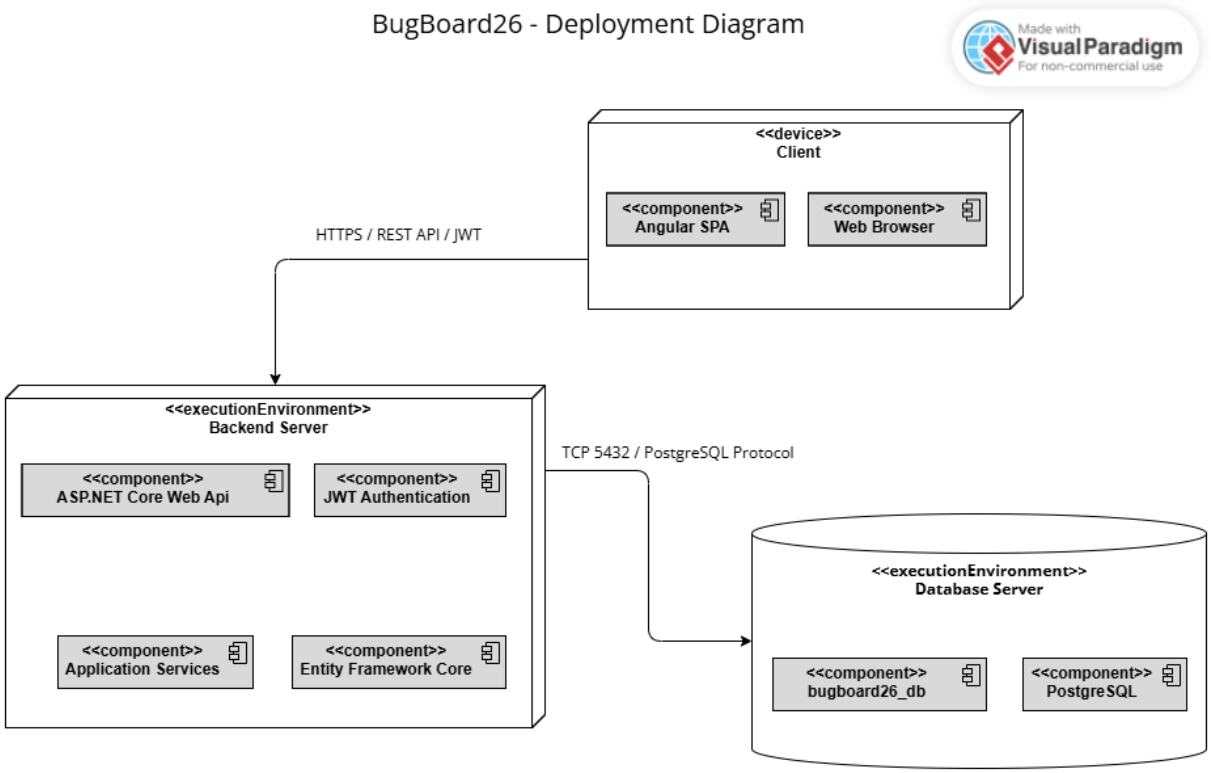


Figura 2.2: Vista logica di deployment preesistente; avvio locale effettivo descritto nel testo.

2.2 Organizzazione Object-Oriented

Elemento	Responsabilita'
Controller	Gestiscono richieste e risposte HTTP, autorizzazioni e coordinamento delle operazioni.
`IssueDomainService`	Logiche semplici di ricerca in memoria, permessi sullo stato e suggerimento assegnatario.
`ApplicationDbContext`	Mapping, relazioni e accesso a PostgreSQL con Entity Framework Core.
`User`, `Issue`, `Notification`	Entita' del dominio con proprieta' e relazioni.
`IPasswordHasher`, `IJwtTokenService`	Contratti per hashing password e generazione dei token; implementazioni iniettate nei controller.
`IssueImageStorage`	Validazione e lettura/scrittura delle immagini in una directory privata.
DTO di richiesta/risposta	Separano i dati delle API dalle entita' persistenti; non espongono password hash o percorsi fisici.

La separazione rende testabili le logiche senza HTTP e consente di cambiare la presentazione mantenendo le API. Le interfacce di autenticazione separano il contratto dall'implementazione. Non sono introdotti repository generici o livelli enterprise: per questo perimetro il DbContext e i servizi presenti sono sufficienti. Alcune query e operazioni sono nei controller; non si dichiara una separazione assoluta di tutta la logica.

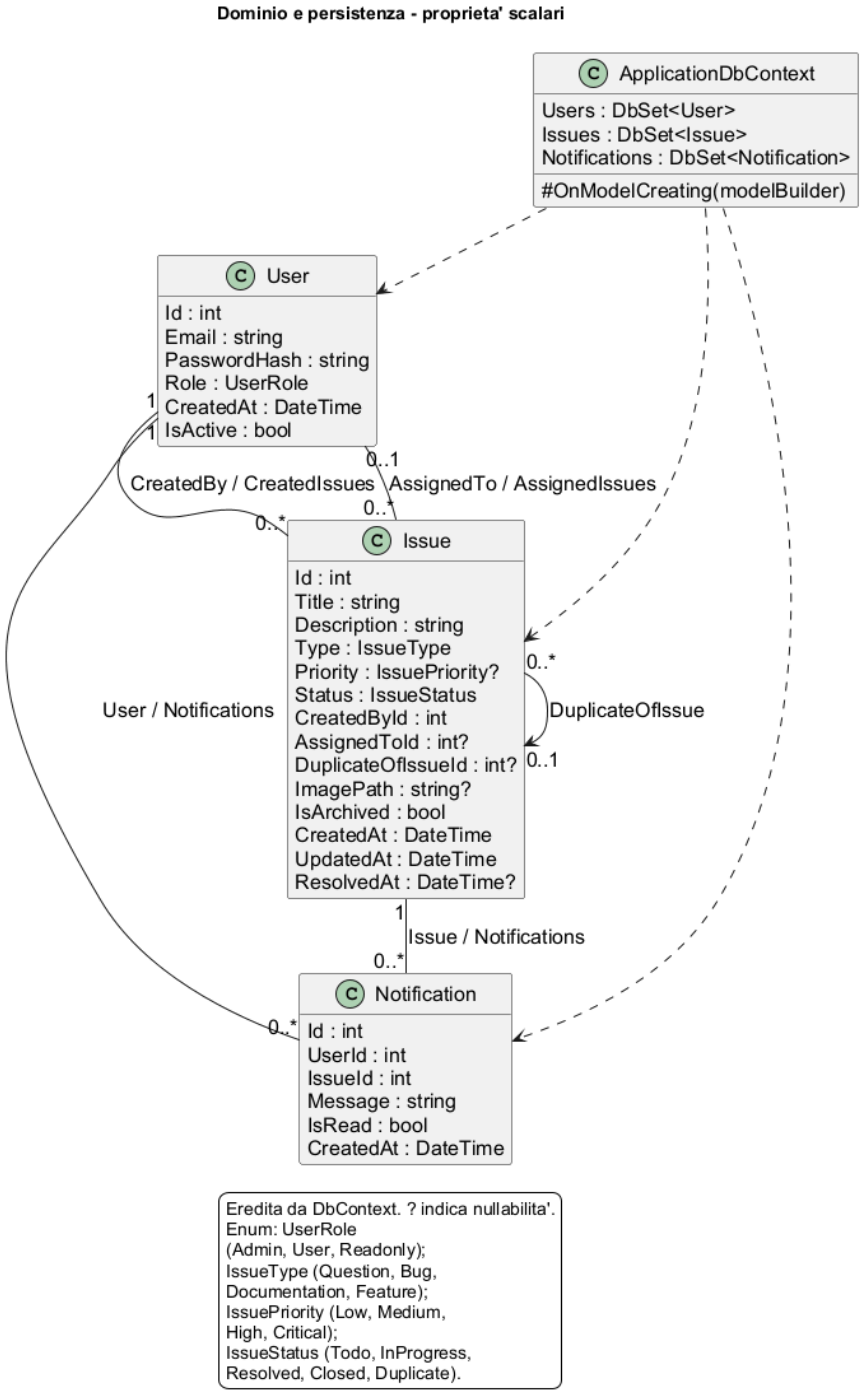


Figura 2.3: Classi di dominio, persistenza e servizi.

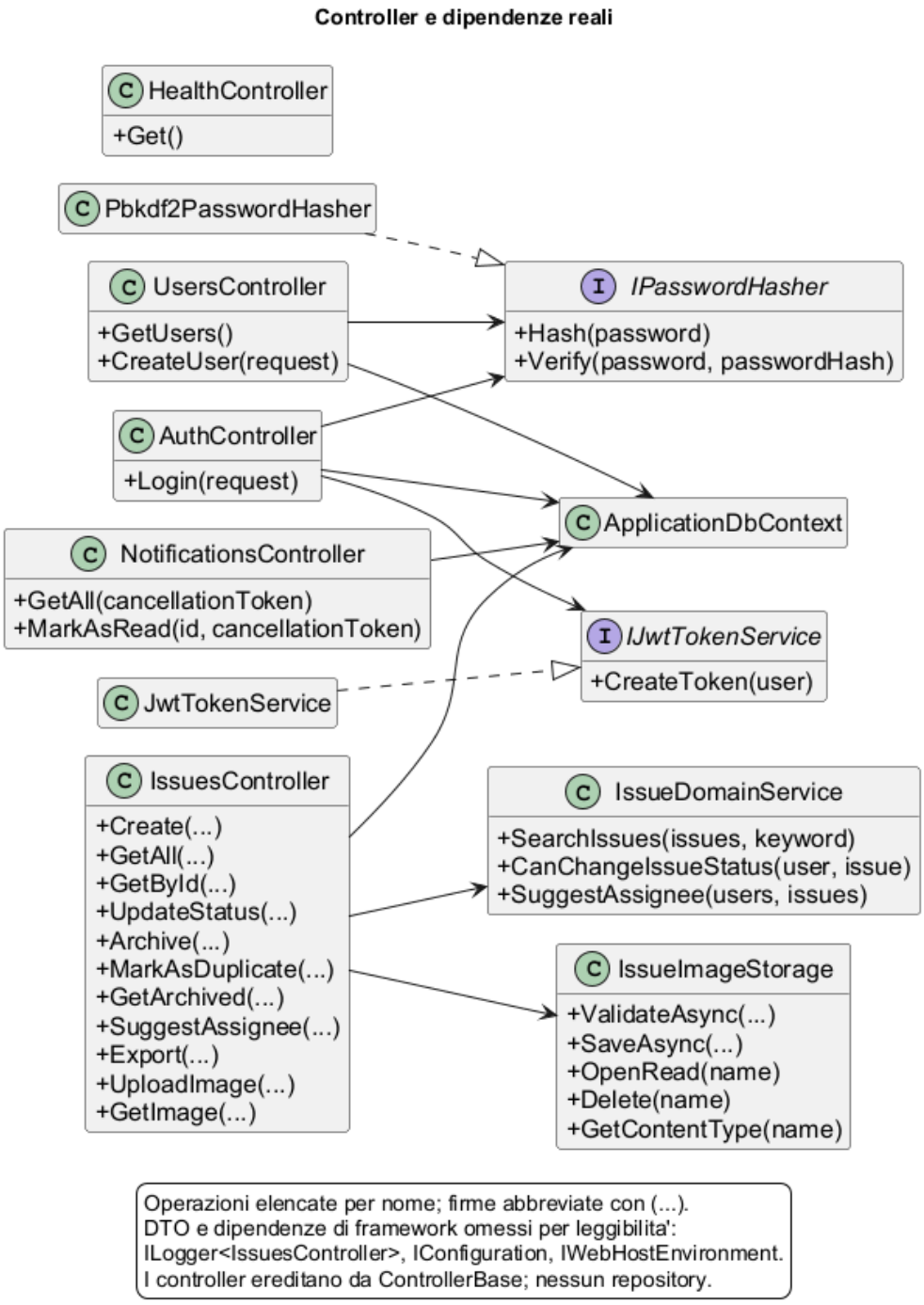


Figura 2.4: Controller e dipendenze.

2.3 Persistenza

Gli identificativi sono interi. Gli enum sono memorizzati come stringhe; le date applicative sono prodotte in UTC.

Entita'	Campi principali
User	Id, Email, PasswordHash, Role, CreatedAt, IsActive. Email univoca; collezioni di issue create/assegnate e notifiche.
Issue	Id, Title, Description, Type, Priority nullable, Status, CreatedById, AssignedToId nullable, DuplicateOfIssueId nullable, ImagePath nullable, IsArchived, CreatedAt, UpdatedAt, ResolvedAt nullable.
Notification	Id, UserId, IssueId, Message, IsRead, CreatedAt.

Ogni issue ha un segnalatore e puo' avere un assegnatario. La relazione con l'originale del duplicato e' un autoriferimento opzionale. Le notifiche appartengono a un destinatario e a una issue. Il mapping impedisce la cancellazione del segnalatore o dell'originale referenziato; la cancellazione dell'assegnatario azzera il riferimento. Non sono esposte nuove funzioni di cancellazione per sfruttare queste regole.

I byte delle immagini non sono in PostgreSQL: il database conserva un nome gestito dal backend. In Docker file e database sono mantenuti in due volumi distinti. Il seed applica le migrazioni e crea l'ADMIN se non esiste gia' un amministratore. Non reimposta la password di account esistenti.

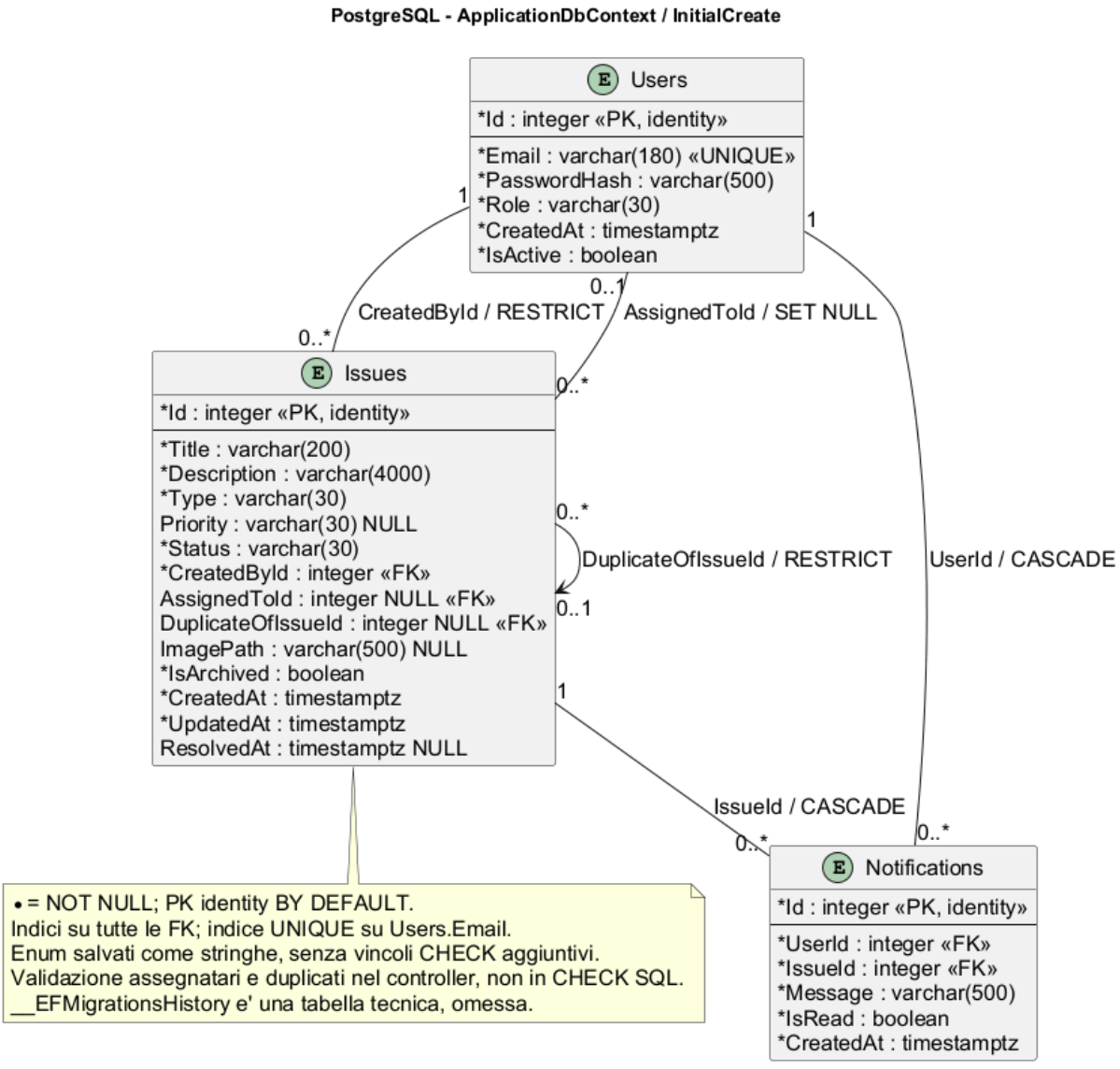


Figura 2.5: Schema della persistenza.

2.4 Autenticazione e autorizzazione

Il login normalizza l'email, verifica account attivo e password con `Pbkdf2PasswordHasher`, quindi restituisce JWT, scadenza e identita' dell'utente. Il frontend conserva la sessione in localStorage, ne controlla la scadenza e invia il token solo alle API del progetto. Un 401 su una richiesta protetta porta al login; un 403 non comporta logout.

I guard e i pulsanti nascosti migliorano l'interfaccia, ma la sicurezza dipende dalle autorizzazioni backend. ADMIN crea utenti, archivia e marca duplicati. USER cambia stato soltanto per le proprie assegnazioni; READONLY non modifica issue. Le notifiche sono consultabili e segnabili come lette soltanto dal destinatario, anche rispetto ad ADMIN.

Il setup locale usa HTTP e credenziali di sviluppo. Un'eventuale esposizione pubblica richiederebbe HTTPS e configurazione adeguata dei segreti; questo documento non certifica un deployment di produzione.

2.5 Ricerca, suggerimento ed export

La lista esclude gli archiviati e applica filtri per tipo, stato, priorita', creatore e assegnatario dove previsti dall'API. La ricerca API usa `ILike` di PostgreSQL su titolo e descrizione. L'helper `SearchIssues` opera invece su collezioni in memoria: i suoi test non sostituiscono i test HTTP della query SQL.

Il suggerimento considera gli utenti attivi ADMIN e USER. Conta soltanto issue assegnate non archiviate in TODO o IN_PROGRESS. Vince il carico minore; a parita', l'ID minore. Nessun candidato produce 404 con messaggio. La consultazione del suggerimento non scrive dati.

Lista ed export riutilizzano la stessa query backend. Il CSV contiene Id, Title, Type, Priority, Status, CreatedByEmail, AssignedToEmail e CreatedAt; usa UTF-8 ed escaping di virgole, doppi apici e ritorni a capo. Il frontend scarica il Blob con nome `issues.csv` e passa i filtri correnti anche durante l'attesa della ricerca live. RxJS `switchMap` annulla le richieste precedenti evitando risultati obsoleti.

2.6 Immagini, archivio e duplicati

L'immagine viene caricata dopo la creazione come multipart con campo `image`. Sono ammessi PNG/JPEG non vuoti entro 5 MiB (5242880 byte). Il backend controlla estensione e marcatori del file, non esegue una decodifica completa o una scansione antivirus. Solo ADMIN o USER creatore puo' caricare; tutti i ruoli autenticati possono leggere. L'immagine viene scaricata tramite HttpClient con JWT e visualizzata con un object URL, revocato quando la pagina viene distrutta.

Un upload fallito non annulla la creazione: il form conserva l'ID e riprova solo il caricamento. L'archiviazione imposta `IsArchived`, senza cancellazione; l'archivio mantiene accessibili dettaglio e immagine. Il duplicato imposta DUPLICATE e `DuplicateOfIssueId`, azzera `ResolvedAt` e non genera la notifica di risoluzione.

2.7 Diagrammi comportamentali

Le viste seguenti descrivono creazione, cambio stato e duplicazione.

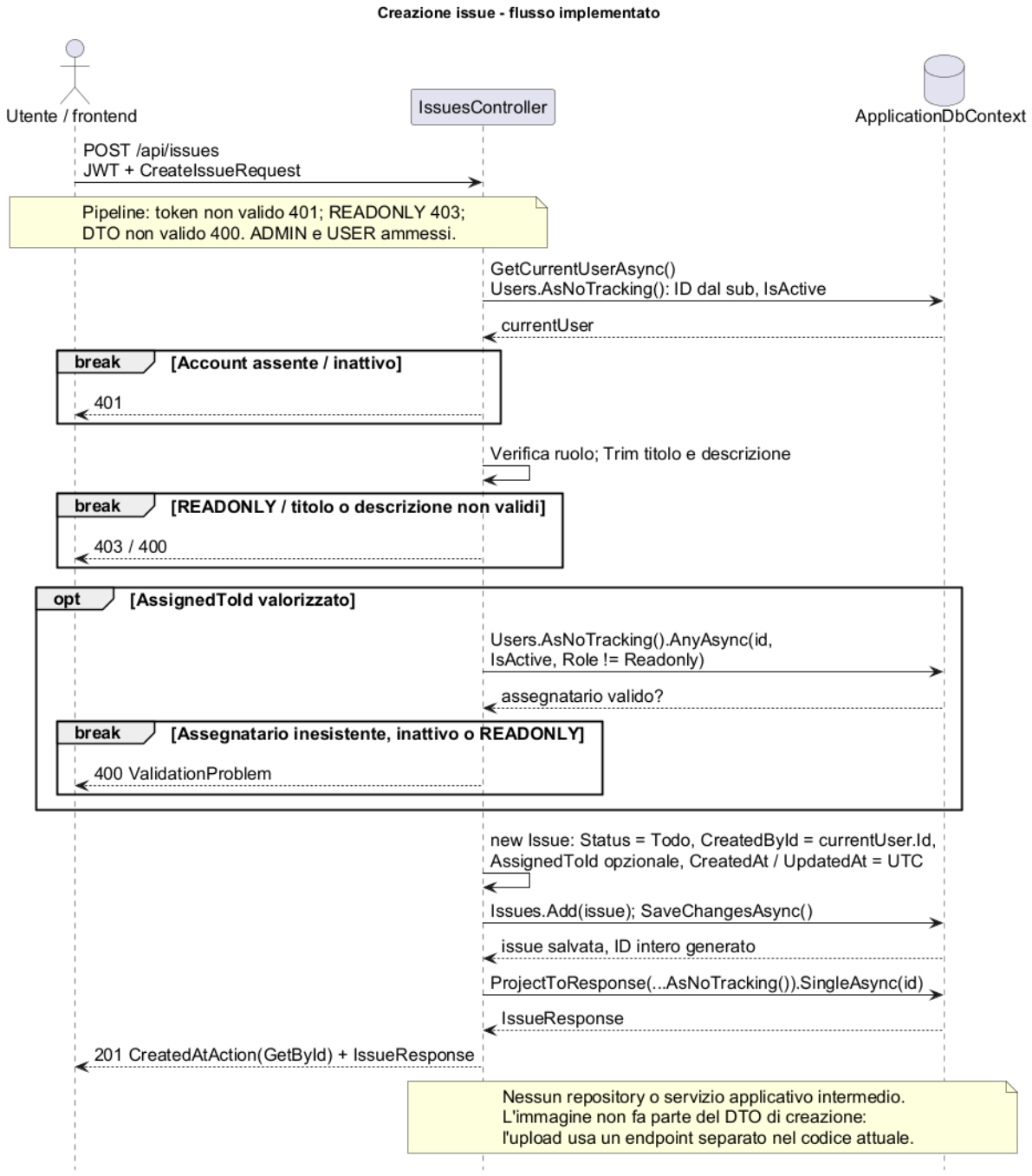


Figura 2.6: Sequenza di creazione della issue.

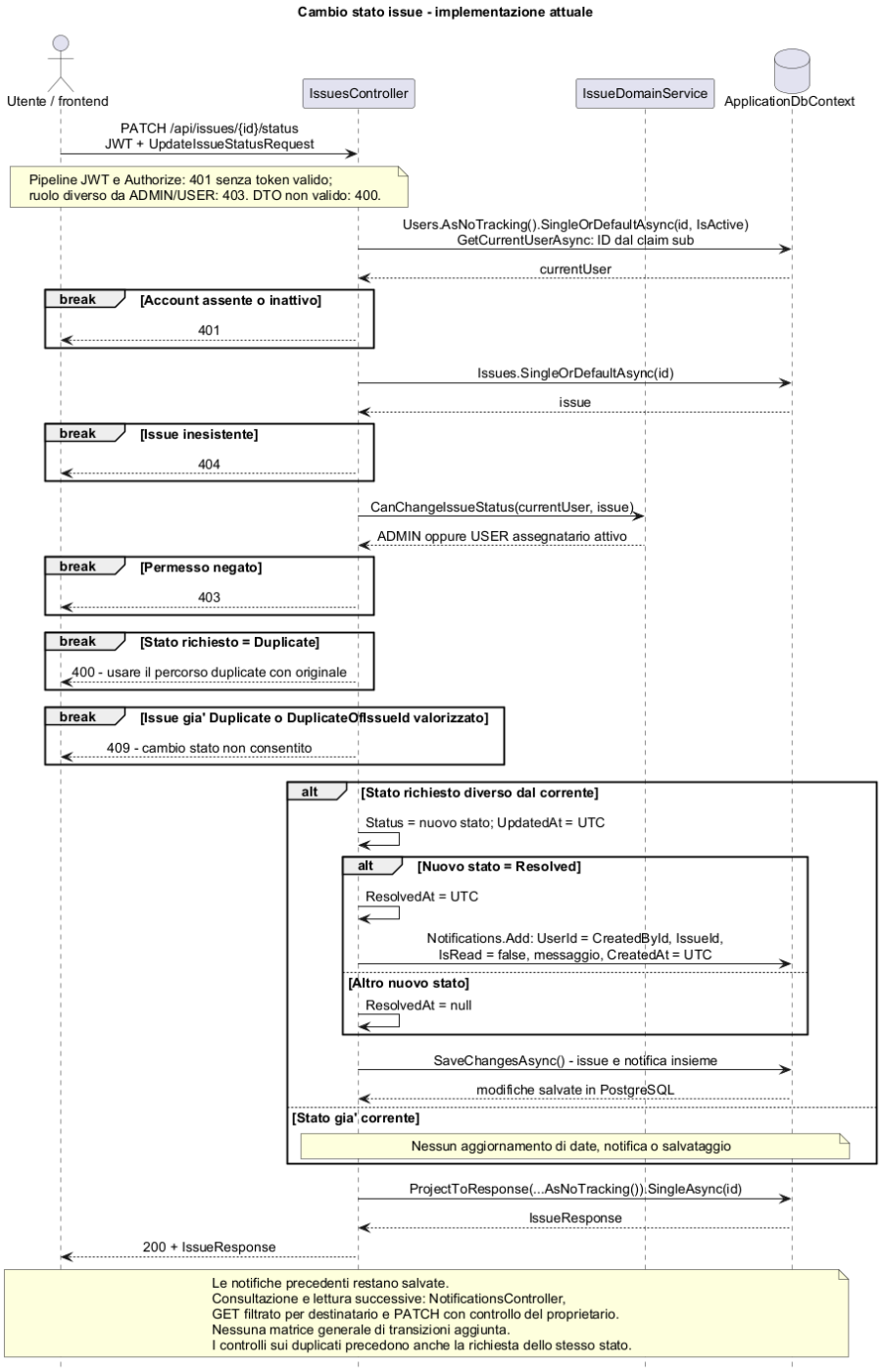


Figura 2.7: Sequenza di cambio stato e notifica.

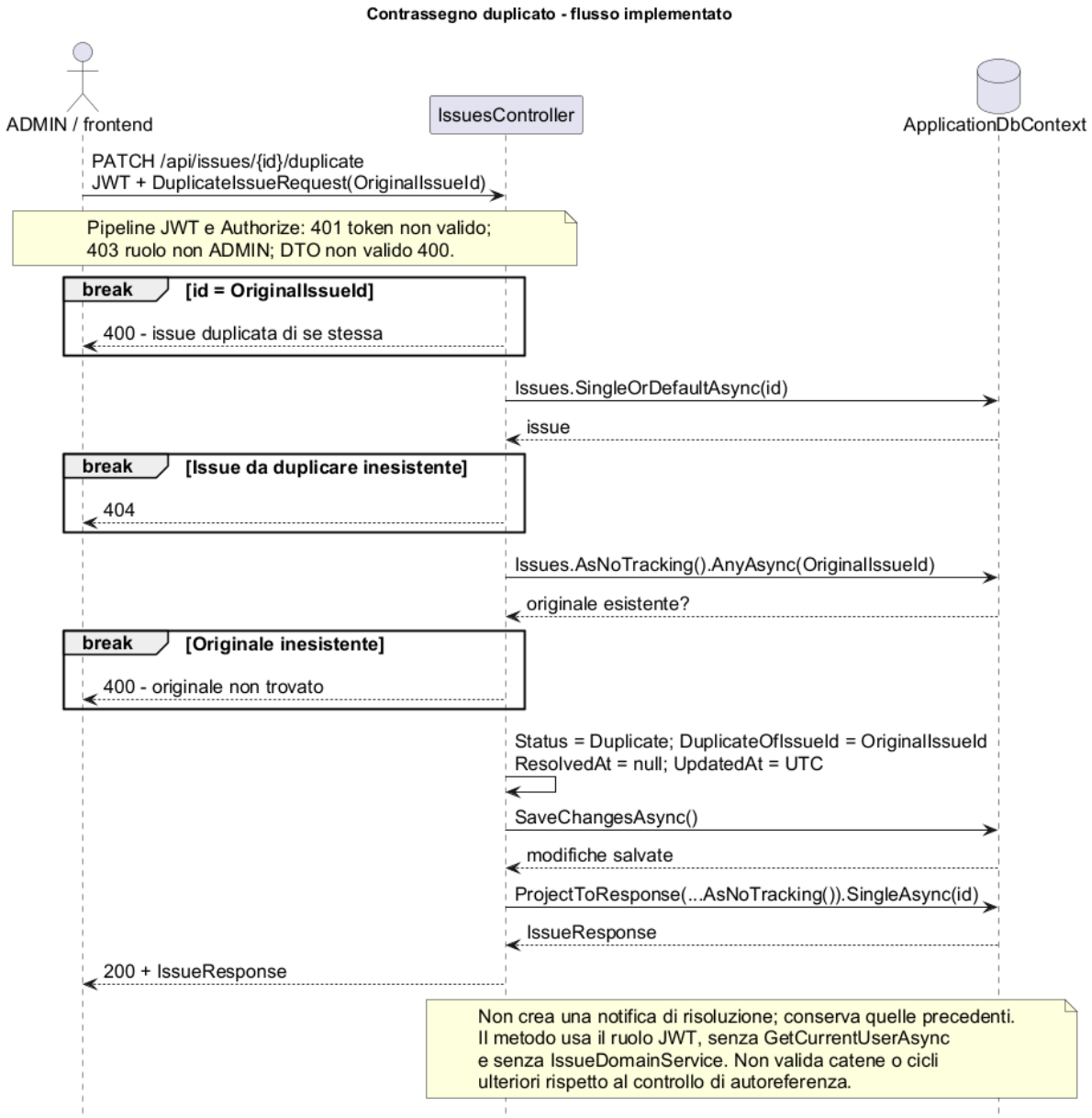


Figura 2.8: Sequenza di contrassegno duplicato.

2.8 Interfacce e avvio

Metodo	Percorso API	Scopo
POST	`/api/auth/login`	Login.
GET / POST	`/api/users`	Lista/creazione utenti, solo ADMIN.
GET / POST	`/api/issues`	Lista/creazione issue.
GET	`/api/issues/{id}`	Dettaglio.
PATCH	`/api/issues/{id}/status`	Cambio stato.
PATCH	`/api/issues/{id}/archive`	Archiviazione ADMIN.
GET	`/api/issues/archived`	Archivio.
PATCH	`/api/issues/{id}/duplicate`	Contrassegno duplicato ADMIN.
GET	`/api/issues/suggest-assignee`	Suggerimento.
GET	`/api/issues/export`	CSV.
POST / GET	`/api/issues/{id}/image`	Upload/lettura immagine.
GET	`/api/notifications`	Notifiche personali.
PATCH	`/api/notifications/{id}/read`	Lettura di una propria notifica.

Le rotte frontend sono `/login`, `/issues`, `/issues/new`, `/issues/:id`, `/issues/archived`, `/users` e `/notifications`. La pagina utenti e' riservata ad ADMIN; la creazione e' protetta anche contro l'accesso diretto READONLY.

Prerequisiti: Docker Desktop con motore Linux, Node.js 22/npm; .NET SDK 10 per i test o per l'avvio backend fuori da Docker. Dalla radice:

```
docker compose -f docker/docker-compose.yml up -d --build
# In un altro terminale:
cd frontend
npm ci
npm start
```

Frontend: http://localhost:4200/login; API: http://localhost:5256/api. Attendere l'avvio delle API prima del login. Su database nuovo l'ADMIN e' `admin@bugboard26.local`, password `Admin123!`: sono credenziali locali di sviluppo. Il README contiene le alternative di configurazione. Non usare `down -v` per conservare i volumi. L'avvio con `dotnet run` usa una directory immagini diversa dal container e non trasferisce automaticamente i file.

2.9 Versionamento e artefatti

Il repository Git e' il riferimento per codice, test e documenti. La cronologia visibile al commit esaminato contiene contributi su autenticazione, issue, frontend, immagini, test e documentazione. Non vengono inventate statistiche di contribuzione o una storia di sviluppo. La release definitiva dovra' identificare la revisione consegnata. I file `.puml` e le immagini sono aggiornati

dalla revisione documentale precedente; i `.vpd` sono storici e non vanno presentati come modelli allineati senza ulteriore verifica CASE.

Capitolo 3

Testing e valutazione dell'usabilita'

3.1 Strategia e livelli di verifica

Le verifiche sono separate in test di dominio, integrazione backend e collaudo frontend con sistema reale. I test di dominio usano xUnit e collezioni in memoria. I test HTTP backend usano WebApplicationFactory con autenticazione reale e PostgreSQL dedicato; non richiedono una porta API esterna. Ogni caso usa un database temporaneo distinto. Il database di sviluppo non viene svuotato dai test.

Il frontend usa Jasmine/Karma. I suoi test simulano servizi o HTTP per isolare il comportamento dei componenti: un test passato non dimostra da solo il collegamento al backend. Il collaudo del 16 ottobre ha invece usato Angular, backend Docker e PostgreSQL reali, con Chrome headless.

3.2 Tre metodi di dominio e classi di equivalenza

Metodo	Casi presenti e motivazione
`SearchIssues` (issues, keyword)	Parola nel titolo, nella descrizione con maiuscole diverse e parola assente: distingue i due campi di ricerca e la classe senza corrispondenze.
`CanChangeIssueStatus` (user, issue)	ADMIN non assegnatario; USER assegnatario/non assegnatario; READONLY anche assegnato; USER inattivo. Separa ruolo, assegnazione e attivazione.
`SuggestAssignee` (users, issues)	Carichi differenti con vincitore alternato, issue risolte/chiuse escluse e nessun candidato assegnabile. Verifica la scelta e il caso limite senza risultato.

I tre metodi hanno ciascuno due parametri. Le Theory rappresentano casi significativi, non un obiettivo numerico. La suite di integrazione aggiunge verifiche di permessi, filtri, escaping CSV, parita' ed esclusioni del suggerimento, immagini e persistenza. La copertura strutturale completa non e' deducibile da questo elenco: va misurata sulla suite completa.

3.3 Test plan – Risoluzione di una issue

Obiettivo: verificare UC-02 e RF-10/RF-11 senza consentire modifiche a utenti non autorizzati. Preparare una issue di prova creata da S e assegnata ad A, entrambi USER attivi e distinti; U e' un altro USER, R e' READONLY. Usare un ADMIN separato. Registrare stato, date e notifiche prima e dopo. Non modificare issue preesistenti per il collaudo.

Caso	Azione	Risultato atteso
TP-01	A imposta RESOLVED.	200; stato e date persistiti; nuova notifica non letta per S.
TP-02	ADMIN non assegnatario aggiorna una issue di prova.	Operazione consentita.
TP-03	U tenta il cambio stato.	Nessun pannello frontend; PATCH diretto 403 senza scritture.
TP-04	R tenta il cambio stato.	Nessun pannello; PATCH diretto 403.
TP-05	Richiesta senza JWT.	401 e nessuna modifica.
TP-06	ID issue inesistente.	404 e nessuna notifica.
TP-07	A ripete lo stato RESOLVED corrente.	Nessuna nuova notifica o variazione delle date.
TP-08	Riapertura e seconda risoluzione.	Data azzerata alla riapertura; nuova data/notifica alla seconda risoluzione.
TP-09	Cambio stato su duplicata.	409; riferimento e stato conservati.
TP-10	S consulta notifiche; U tenta lettura della notifica di S.	S vede la propria notifica; U non la vede e PATCH read altrui restituisce 403.

Criterio di accettazione: tutti i casi pianificati eseguiti con esito registrato e nessuna modifica non autorizzata. I test automatici presenti coprono una parte del piano; il piano non viene dichiarato interamente eseguito. TP-08 richiede ancora esito finale dedicato.

3.4 Collaudo frontend reale – 16 ottobre 2026

Riferimento: commit `e13995140dff382dc3d548208c6c1e60be91fe5d`, checkout pulito prima e dopo. API sulla porta 5256 e Angular sulla 4200; PostgreSQL del Compose di sviluppo. Il precedente blocco Docker e' stato risolto avviando Docker Desktop; non e' stato necessario cambiare codice.

Sono stati creati tre account con prefisso `collaudo-20261016084339` (due USER e un READONLY) e le issue 4–7. Le operazioni di modifica hanno riguardato soltanto queste nuove issue. La issue 4 e' stata duplicata e archiviata; la 7, predisposta tramite API per avere un assegnatario deterministico, e' stata risolta dal relativo USER. Le altre creazioni sono passate dal form frontend. Non sono riportati token o credenziali dei partecipanti nei risultati.

Area	Esito osservato
Login e guard	Login ADMIN riuscito; password errata: 401 e messaggio UI. Accesso anonimo a creazione reindirizzato al login; nessuna navbar nella login.

Area	Esito osservato
Ricerca e lista	Ricerca live, digitazione rapida con risultato finale corretto, filtri tipo/stato/priorita', ordinamento per titolo, lista vuota e reset completo riusciti.
Export	Download reale; ID e ordine nel CSV uguali alla lista filtrata.
Creazione	Issue senza immagine e con PNG create; immagine scaricata autenticata e visualizzata.
Recupero upload	JPEG volutamente non valido rifiutato dall'API; ID conservato; JPEG valido caricato al secondo tentativo senza seconda creazione.
Validazione	Titolo/descrizione di soli spazi: nessuna POST e messaggi sui campi. Formato non ammesso e file oltre 5 MiB rifiutati dal frontend.
Stato e ruoli	USER assegnatario risolve; stato persistito. USER non assegnatario senza pannello. USER non assegnatario e READONLY ricevono 403 dal PATCH diretto.
READONLY	Nessuna azione di modifica nel dettaglio; accesso diretto a creazione reindirizzato; immagine consultabile.
Duplicato e archivio	ADMIN marca duplicato con collegamento aggiornato; archivia; issue assente dalla lista principale e dettaglio apribile dall'archivio.
Errori e layout	ID inesistente mostra "Issue non trovata". Lista, form e dettaglio controllati a 1366 e 390 pixel senza overflow orizzontale; screenshot ispezionati.
Notifica	Via API verificata la notifica di risoluzione della issue 7 al segnalatore ADMIN; non un collaudo completo della pagina notifiche.

La build frontend e' riuscita e i 27 test Jasmine/Karma sono passati senza watch con ChromeHeadless. Alcune asserzioni dello script temporaneo sono state corrette per attendere il rendering o cercare il testo effettivo; le prove sono state ripetute. Nessuna correzione applicativa e' risultata necessaria. Gli screenshot sono stati prodotti nella cartella temporanea locale; non sono allegati permanenti della repository.

Questa prova non misura le prestazioni con rete rallentata o grandi quantita' di dati, non copre ogni browser e non sostituisce il collaudo di persistenza dopo ricreazione dei container.

3.5 Esecuzione ripetibile e qualita' backend

Dalla radice del repository, con Docker disponibile:

```
docker compose -p bugboard26-tests ‘
  -f backend/BugBoard26.Tests/docker-compose.tests.yml up -d --wait
dotnet test backend/BugBoard26.slnx
# Frontend, dalla cartella frontend:
npm run build
node node_modules/@angular/cli/bin/ng.js test ‘
  --watch=false --browsers=ChromeHeadless
```

Il database di test usa la porta 55432 ed e' distinto da quello di sviluppo. La sola esecuzione dei test di dominio con filtro non sostituisce la suite di integrazione. Il conteggio backend definitivo sulla revisione finale resta da registrare dal collega responsabile.

SonarQube locale e la raccolta OpenCover sono predisposti in `quality/analyze-backend.ps1` e `quality/coverage.runsettings`. Non e' ancora presente un report Sonar effettivo con importazione della coverage e Quality Gate verificati. La prova storica del 9 ottobre riguarda solo i test di dominio e non rappresenta la copertura totale. Il collaudo del 16 ottobre ha confermato in build l'avviso NU1903 sulla dipendenza transitiva `Microsoft.OpenApi` 2.0.0: il collega backend deve valutarlo; non si dichiara risolto o risultato di Sonar.

3.6 Valutazione dell'usabilita' – piano da eseguire

Il collaudo tecnico precedente non e' una prova con utenti potenziali. La traccia richiede sia una review sistematica sia osservazioni e survey con utenti. Non sono disponibili sessioni o questionari compilati; questa sezione definisce il piano, non risultati fittizi.

3.6.1 Review tramite checklist

Controllare e registrare per ogni voce: esito, pagina, problema osservato e possibile intervento limitato al perimetro.

- Terminologia coerente fra lista, form e dettaglio.
- Navigazione riconoscibile e ritorno alla lista disponibile.
- Campi obbligatori ed errori comprensibili vicino al campo.
- Caricamento, esito del salvataggio, errore e lista vuota distinguibili.
- Azioni coerenti con il ruolo, senza pulsanti di modifica per READONLY.
- Uso da tastiera, focus visibile e leggibilita' su schermi piccoli.
- Recupero dell'upload senza indurre una seconda creazione.

3.6.2 Sessioni con utenti e questionario

Coinvolgere utenti reali o potenziali del sistema, usando account e dati di prova. Annotare un codice anonimo del partecipante, ruolo, attivita', completamento autonomo/con aiuto, errori e osservazioni. Non raccogliere dati personali non necessari. Il gruppo deve osservare senza suggerire subito la soluzione e registrare le difficolta' effettive.

Attivita' proposte: segnalare una issue; cercarla e filtrarne l'elenco; aprire il dettaglio e risolverla con un account assegnatario; consultare una issue con account READONLY. Le azioni ADMIN possono essere valutate con un partecipante che rappresenti quel ruolo.

Al termine, raccogliere valutazioni da 1 (per nulla) a 5 (molto) su: chiarezza della navigazione, comprensibilita' dei campi, facilita' nel trovare una issue, chiarezza degli esiti e dei permessi. Aggiungere una domanda aperta: "Quale passaggio e' stato meno chiaro?". Riportare numero reale di sessioni, risultati e limiti del campione; non attribuire validazione statistica a poche osservazioni.

Capitolo 4

Limiti e completamento della consegna

4.1 Punti ancora aperti

1. Eseguire e registrare suite backend completa e coverage sulla revisione finale; integrare il report SonarQube effettivo.
2. Verificare dati e immagini dopo riavvio e ricreazione dei container conservando entrambi i volumi.
3. Eseguire review e sessioni di usabilita', sostituendo il piano con risultati realmente raccolti.
4. Chiarire RF-17 rispetto alla lettura delle proprie notifiche READONLY. Il codice attuale la consente; questo documento non approva una nuova eccezione.
5. Chiudere la conformita' CASE: le immagini aggiornate provengono dai sorgenti PlantUML; i file Visual Paradigm storici non rappresentano tutte le modifiche attuali.
6. Valutare l'avviso NU1903 confermato durante la build backend.

Non sono previsti paginazione, ripristino archivio, refresh token o gestione immagini dal dettaglio frontend. Non vengono aggiunti come nuovi requisiti. Il piano di consegna non richiede nuove funzionalita' fuori assegnazione.

4.2 Pacchetto e discussione

La consegna deve identificare repository e release definitiva, includere sorgenti, file di build/deployment, test, documento unificato, diagrammi, report e presentazione. Non includere dipendenze installate, file bin/obj, token, segreti personali o risultati fittizi. Ogni componente consegna i link tramite Teams secondo la traccia e comunica ai docenti l'invio.

La presentazione tecnica dura al massimo dieci minuti ed e' seguita dalla demo. Deve spiegare scelte architetturali, modello, autorizzazioni e test, non soltanto elencare funzioni. Tutti i membri devono conoscere il sistema. Sequenza demo proposta: login, creazione con immagine, ricerca/export, risoluzione come assegnatario e notifica al segnalatore, duplicato/archivio come ADMIN, consultazione READONLY. Usare dati nuovi e riconoscibili.

4.3 Manutenzione del documento

Compilare dalla directory che contiene questo sorgente per risolvere i percorsi relativi delle figure. Per esempio, con Tectonic disponibile:

```
tectonic documentazione_finale.tex
```

Conservare questo sorgente insieme ai mockup e diagrammi referenziati; il PDF puo' invece essere consultato autonomamente. Prima della consegna aggiornare versione, commit, risultati delle verifiche ancora aperte e stato del documento, senza riscrivere i requisiti per giustificare il codice.